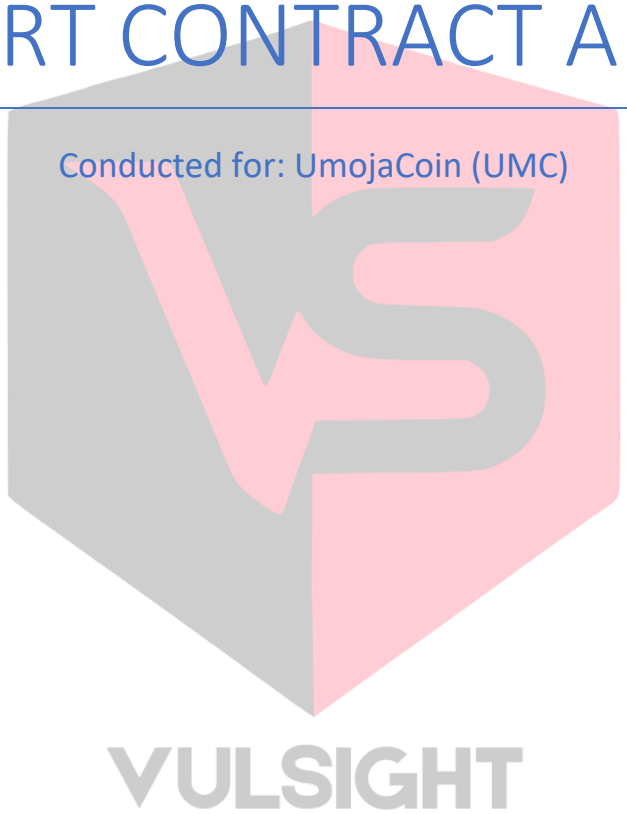
SMART CONTRACT AUDIT

Conducted for: UmojaCoin (UMC)



VULSIGHT
12/12/2024

Contents

Executive Summary:

Our firm was engaged to conduct a follow-up security audit of the UmojaCoin (UMC) smart contract's V2 implementation. This audit follows our previous assessment and evaluates both the remediation of prior findings and identifies any new security concerns.

The V2 contract successfully addressed all previously identified issues from our initial audit, including:

- Implementation of the listing time mechanism
- Implementing the whitelist functionality correctly
- Correction of token distribution implementation

However, our current audit revealed two new critical vulnerabilities in the token vesting implementation:

- An unbounded vesting calculation that allows beneficiaries to extract tokens beyond their allocated amount (Critical)
- A mathematical error in the monthly release rate calculation that results in incorrect token distribution (Critical)

These newly identified vesting mechanism issues require immediate attention before mainnet deployment to prevent potential token distribution anomalies.

Project Background:

The smart contract is written in solidity. The token contract is currently deployed on the Binance Smart Chain Testnet.

Audit Scope:

Deployed Addresses	**0x994368ec82274d9FBD41597Dca44Fb6C049278Ac**
Chain	**Binance Smart Chain Testnet**
Language	**Solidity**
Audit Completion Date	**12/12/24**

Contract Summary:

- **Contract type:** The ***UmojaCoin*** is an upgradeable ERC20 token contract that implements OpenZeppelin's upgradeable extensions for enhanced functionality and security.
- **Token standard implementation:** The contract implements the upgradeable ERC20 standard and includes UUPS (Universal Upgradeable Proxy Standard) upgradeability along with AccessControl for role-based permissions.
- **Token specifications:** The token has the symbol "UMC" (UmojaCoin) with a total supply of 1.5 billion tokens, distributed across different allocations.
- **Access control:** The contract uses OpenZeppelin's ***AccessControl*** pattern with an ***ADMIN_ROLE*** that can perform administrative functions like token allocation and whitelist management.
- **Vesting mechanism:** Implements a vesting system with two categories (12-months and 24-months).

We found:

- **2 Critical risk findings**
- **0 High risk findings**
- **0 Medium risk findings**
- **0 Low risk findings**

Severity Definitions:

Risk Level	Description
Critical	**Any kind of vulnerability that could lead to direct token or monetary loss**
High	**Any type of vulnerability that could disrupt the proper functioning of smart contract or indirectly lead to token or monetary loss.**
Medium	**Any type of vulnerability that could cause undesired actions but no serious disruption or monetary loss**
Low	**Any type of vulnerability that doesn't have any significant impact on the execution of the proper functioning of contract**

Technical Checks:

Checks	Result
Solidity version not specified	Passed
Solidity version too old	Passed
Integer overflow/underflow	Passed
Function input parameters check bypass	Passed
Insufficient randomness used	Passed
Fallback function misuse	Passed
Race Condition	Passed
Matching Project Specifications	Not Passed
Logical Vulnerabilities	Not Passed
Function visibility not explicitly declared	Passed

Use of deprecated keywords/functions	Passed
Out of Gas issue	Passed
Front Running	Passed
Insufficient Decentralization	Passed
Function input parameters lack of check	Passed
Proper function Access Control	Passed
Hardcoded Data	Passed

Code Quality and Documentation:

The audit scope included one smart contract, which was written in Solidity programming language. The code is well indented and clear in nature.

Previous Findings:

Missing Listing Time Implementation:

The contract lacks a critical vesting control mechanism specified in the documentation. According to the specifications, there should be a ***setListingTime*** function that allows the admin to set the starting time for all vesting schedules. This function is completely absent from the implementation.

Instead of using a configurable listing time as the reference point for vesting calculations, the contract currently uses ***block.timestamp*** directly in the ***allocateTokens*** function:

```
vestingSchedules[_beneficiary] = VestingInfo({
    amount: _amount,
    startTimestamp: block.timestamp,  // Uses block.timestamp instead of listing time
    duration: duration,
    released: 0,
    category: category
});
```

This deviation from the specifications has several implications:

1. Each allocation has its own independent start time rather than a synchronized vesting schedule
2. The admin cannot control or coordinate the start of the vesting period
3. There's no way to delay the start of vesting until token listing
4. Different users allocated tokens at different times will have desynchronized vesting schedules

Current Status:

The vulnerability was found to be fixed.

Unused Whitelisting Mechanism:

The contract implements a whitelist mechanism through the ***whitelisted*** mapping and ***setWhitelist*** function, but this functionality is never actually used in any operational context:

```
mapping(address => bool) public whitelisted;

function setWhitelist(address account, bool status) external onlyAdmin {
    whitelisted[account] = status;
    emit Whitelisted(account, status);
}
```

The presence of unused code:

1. Increases the contract's deployment cost unnecessarily
2. Could create confusion about the actual security model of the contract
3. May mislead integrators about available functionality

Current Status:

The vulnerability was found to be fixed.

Token Distribution Implementation Deviates from Specifications:

The contract's token distribution implementation does not match the specified allocation requirements. According to the documentation, presale tokens should be allocated to the admin wallet, but the current implementation mints them to the treasury wallet instead. The current implementation:

```
function initialize(address admin) public initializer {
    // ... initialization code ...

    _mint(TREASURY_WALLET, EARLY_STAGE_TOKENS);
    _mint(TREASURY_WALLET, PRESALE_TOKENS);  // Incorrect: Should go to admin
    _mint(TREASURY_WALLET, COFOUNDERS_TOKENS);
    _mint(DEVELOPER_WALLET, DEVELOPERS_TOKENS);
    _mint(MARKETING_WALLET, MARKETING_TOKENS);
    _mint(TREASURY_WALLET, FREE_FOR_TRADING_TOKENS);
}
```

Specified Distribution:

1. Treasury Wallet: Early Stage, Co-founders, Free Trading tokens (1,083,700,000 tokens)
2. Admin Wallet: Presale tokens (100,000,000 tokens)
3. Developer Wallet: Developer tokens (250,000,000 tokens)
4. Marketing Wallet: Marketing tokens (66,300,000 tokens)

This mismatch results in:

1. Treasury wallet receiving excess tokens beyond specified allocation
2. Admin wallet not receiving allocated presale tokens
3. Deviation from the documented token distribution model

Current Status:
The vulnerability was found to be fixed.

New Findings:

Critical findings:
Two critical findings were found in the smart contract.

Unbounded Token Vesting Leads to Excess Token Release:
A critical vulnerability exists in the token vesting calculation where the ***calculateVestedAmount*** function lacks proper duration bounds, allowing beneficiaries to receive more tokens than their original allocation. The vesting schedule continues to release tokens beyond 100% of the allocated amount due to a flawed calculation that applies fixed percentage releases (15.67% for 12-month and 5.22% for 24-month vesting) without checking the total vesting duration.

For example, after 30 months:

- 12-month vesting category releases 356.68% of original allocation
- 24-month vesting category releases 123.4% of original allocation

The vulnerability can be exploited by calling the ***releaseVestedTokens()*** function after the intended vesting period, allowing the beneficiary to extract more tokens than initially allocated.

Code snippet of vulnerable calculation:

```
uint256 remainingTokens = totalAmount - ((totalAmount * 7) / 100);
uint256 monthlyRelease = (remainingTokens * 1567) / 10000; // 15.67% for 12-month
// or (remainingTokens * 522) / 10000; // 5.22% for 24-month
return ((totalAmount * 7) / 100) + (monthlyRelease * additionalMonths);
```

Remediation:
The ***monthsElapsed*** value should be capped to have an upper bound.

Incorrect Base Amount in Vesting Rate Calculation:
The vesting calculation contains a mathematical error in how the monthly release rate is applied. The formula uses 15.67% (for 12-month vesting) of the remaining 93% tokens, when it should use 15.5% of the total amount to properly distribute the remaining 93% over 6 months.

For example:

- 12-month schedule delivers only 94.4% of tokens by month 12 (e.g. 944,386 tokens out of 1,000,000)
- 24-month schedule delivers only 94.38% of tokens by month 24 (e.g. 943,828 tokens out of 1,000,000)

The issue occurs because the calculation applies two reductions:

- First reducing to remaining tokens (93%)
- Then applying the monthly percentage to this reduced amount.

Code snippet of vulnerable calculation (similar for 24 months case):

```
remainingTokens = totalAmount - ((totalAmount * 7) / 100); // Get 93% of tokens
monthlyRelease = (remainingTokens * 1567) / 10000; // 15.67% of the 93%
```

This leads to:

- Monthly release = 15.67% of 93% = 14.57% of total amount
- 6 months = 14.57% * 6 = 87.44% of original tokens
- Total with initial 7% = 94.44%

Remediation:
Calculate the monthly release based on the total amount.

High Findings:

Zero high findings were found in the smart contract.

Medium Findings:

Zero medium findings were found in the smart contract..

Low findings:

Zero low findings were found in the smart contract.

Disclaimer:

The smart contract was tested on a best-effort basis. The smart contract was analyzed with the best security practices known at the time of writing this report. Due to the fact that the total number of test cases is unlimited and the fact that new vulnerabilities in technologies are discovered every day, the audit report makes no guarantees on the security of the contract. It is recommended to not just rely on this audit report alone. A bug bounty program for this smart contract may be created to identify any vulnerabilities within the future.